

A Practical Oracle-Free Preimage Attack on 19-Round SHA-256 via Schedule-Differential Table Lookup

Kameldip Singh Basra

Independent Researcher

kameldipbasra@gmail.com

June 2026

Abstract

We present the first practical oracle-free preimage attack on 19-round SHA-256. Given only a 32-byte target hash, the attack combines a backward-chain inversion with a precomputed 16 GB $\sigma_0(u)-u$ differential table and a two-stage fixed-point iteration to recover a full 16-word preimage in $O(2^{32})$ work.

The central algebraic insight is a cancellation identity we call the *W9 differential*: when the message word W_9 is initialised with $a_2 = a_3 = 0$ (producing an approximation $\hat{W}_9$), the resulting error satisfies $\hat{W}_9 - W_9 = a_1$ exactly. This identity causes a_1 to cancel in the W_{16} schedule-consistency equation, reducing the C0 filter to a single table lookup per a_0 candidate. Analogous cancellations in the W_{17} and W_{18} constraints eliminate a_2 and a_3 from the respective lookup targets, enabling a fixed-point iteration that converges with empirical probability $\approx 2.3 \times 10^{-5}$ per C0 survivor. A final W_9 bit-filter with probability 2^{-32} gates forward verification.

Three independent concrete preimages were found and verified on a single NVIDIA H100 GPU, with observed times of 14.5 minutes, 8.5 minutes, and 1.4 minutes at a total GPU cost under \$1.50. This advances the practical oracle-free preimage frontier by at least one round beyond the prior 18-round record.

Full SHA-256 (64 rounds) is not threatened; the result is relevant as evidence of how algebraic structure in the message schedule permits systematic cancellation of unknowns.

Keywords: SHA-256, preimage, oracle-free cryptanalysis, reduced-round attack, schedule-differential table, message-schedule inversion.

1 Introduction

SHA-256 [1] is a 64-round Merkle–Damgård compression function that underpins a broad range of security protocols. Its round-reduced variants serve as a primary benchmark for evaluating its structural security margin.

Attack model. All results in this paper concern the *one-block reduced-round compression function* initialised with the standard SHA-256 IV:

$$H_R(W_0, \dots, W_{15}) = \text{IV} + \text{compress}_R(\text{IV}, W_0, \dots, W_{15}).$$

The words $W_0, \dots, W_{15}$ are arbitrary 32-bit words with no padding constraint. We make *no claim* about standard length-padded SHA-256. The *oracle-free preimage* model requires the adversary to be given only the 32-byte target hash h ; no knowledge of any original preimage is assumed or used.

Prior work. Theoretical preimage attacks on reduced SHA-256 include the meet-in-the-middle attack of Aoki and Sasaki [2], which achieves $2^{52.3}$ for a pseudo-preimage and $2^{104.5}$ for a preimage at 24 rounds, but has never been executed concretely. Collision attacks have been pushed to 24 steps [4], but these do not yield preimages. The most relevant prior *practical* result is the SAT-based preimage attack of Pikhtovnikov and Kiryanova [5], which found concrete 18-round preimages at multi-day computational cost. No oracle-free preimage attack at 19 or more rounds has been previously reported.

This work. We present a practical oracle-free preimage attack on $R = 19$ rounds. The key contributions are:

1. The *W9 differential identity* (Lemma 1): a structural property of the SHA-256 round equations that causes a_1 to cancel algebraically in the C0 schedule constraint, enabling a table-based lookup over the full 2^{32} space of a_0 values.
2. Analogous *unknown-cancellation identities* for a_2 (via the W_{17} constraint) and a_3 (via the W_{18} constraint), together forming a fixed-point iteration (Proposition 2).
3. Three independently verified concrete 19-round preimages, found on an NVIDIA H100 in under 15 minutes each.

2 SHA-256 Preliminaries

Notation. All arithmetic is modulo 2^{32} unless stated otherwise. We write $\oplus$ for bitwise XOR, $\&$ for bitwise AND, $\bar{x}$ for bitwise NOT. Rotations and shifts of a 32-bit word x :

$$\text{ROTR}^n(x) = (x \gg n) \mid (x \ll (32 - n)), \quad \text{SHR}^n(x) = x \gg n.$$

Round functions.

$$\begin{aligned} \Sigma_0(a) &= \text{ROTR}^2(a) \oplus \text{ROTR}^{13}(a) \oplus \text{ROTR}^{22}(a), \\ \Sigma_1(e) &= \text{ROTR}^6(e) \oplus \text{ROTR}^{11}(e) \oplus \text{ROTR}^{25}(e), \\ \sigma_0(x) &= \text{ROTR}^7(x) \oplus \text{ROTR}^{18}(x) \oplus \text{SHR}^3(x), \\ \sigma_1(x) &= \text{ROTR}^{17}(x) \oplus \text{ROTR}^{19}(x) \oplus \text{SHR}^{10}(x), \\ \text{Ch}(e, f, g) &= (e \& f) \oplus (\bar{e} \& g), \\ \text{Maj}(a, b, c) &= (a \& b) \oplus (a \& c) \oplus (b \& c). \end{aligned}$$

State labels. We write the 8-word compression state as $(a_r, a_{r-1}, a_{r-2}, a_{r-3}, e_r, e_{r-1}, e_{r-2}, e_{r-3})$ after round r , with IV boundary values $a_{-4}, \dots, a_{-1}$ and $e_{-4}, \dots, e_{-1}$:

$$a_{-1}=\text{IV}[0], \ a_{-2}=\text{IV}[1], \ a_{-3}=\text{IV}[2], \ a_{-4}=\text{IV}[3], \ e_{-1}=\text{IV}[4], \ e_{-2}=\text{IV}[5], \ e_{-3}=\text{IV}[6], \ e_{-4}=\text{IV}[7].$$

Round equations.

$$T_1^{(r)} = \Sigma_1(e_{r-1}) + \text{Ch}(e_{r-1}, e_{r-2}, e_{r-3}) + e_{r-4} + K_r + W_r, \tag{1}$$

$$T_2^{(r)} = \Sigma_0(a_{r-1}) + \text{Maj}(a_{r-1}, a_{r-2}, a_{r-3}), \tag{2}$$

$$a_r = T_1^{(r)} + T_2^{(r)}, \tag{3}$$

$$e_r = a_{r-4} + T_1^{(r)}. \tag{4}$$

Note that e_{r-4} in (1) is the *h-register* word (boundary value $e_{-4} = \text{IV}[7]$), while a_{r-4} in (4) is the *d-register* word (boundary value $a_{-4} = \text{IV}[3]$); these are distinct IV words and distinct chain values for all $r \geq 4$.

From (3)–(4): $T_1^{(r)} = a_r - T_2^{(r)}$, and rearranging (1):

$$W_r = T_1^{(r)} - \Sigma_1(e_{r-1}) - \text{Ch}(e_{r-1}, e_{r-2}, e_{r-3}) - e_{r-4} - K_r. \quad (5)$$

Message schedule. For $r \geq 16$ the message schedule expands:

$$W_r = \sigma_1(W_{r-2}) + W_{r-7} + \sigma_0(W_{r-15}) + W_{r-16}. \quad (6)$$

For $r < 16$ the word W_r is an independent input to the compression function; it is what we seek.

Hash output. After R rounds the hash is $h[i] = \text{IV}[i] + s_i$ for $i = 0, \dots, 7$, where the final state vector is $(s_0, \dots, s_7) = (a_{R-1}, a_{R-2}, a_{R-3}, a_{R-4}, e_{R-1}, e_{R-2}, e_{R-3}, e_{R-4})$.

3 Structural Analysis

3.1 Backward Chain

From the R -round hash we recover the final state directly:

$$a_{R-1}, a_{R-2}, a_{R-3}, a_{R-4}, e_{R-1}, e_{R-2}, e_{R-3}, e_{R-4}.$$

We then extend backward using $T_2^{(r)} = \Sigma_0(a_{r-1}) + \text{Maj}(a_{r-1}, a_{r-2}, a_{r-3})$, $T_1^{(r)} = a_r - T_2^{(r)}$, $a_{r-4} = e_r - T_1^{(r)}$, iterating for $r = R-1, R-2, R-3, R-4$ to recover $a_{R-5}, \dots, a_{R-8}$.

For $R = 19$ this yields $\{a_{11}, a_{12}, \dots, a_{18}\}$: the state words a_{11} through a_{18} are determined by the target hash alone, with no knowledge of any preimage.

3.2 Free Parameters and Schedule Constraints

At $R = 19$ the unconstrained parameters are $a_0, \dots, a_{10}$ (11 words). Of these, $a_4, \dots, a_{10}$ are chosen uniformly at random per attempt (*context*), forming 7 attacker-controlled free words. The remaining $a_0, \dots, a_3$ are the targets of the attack phases.

The schedule (6) first activates at $r = 16$, yielding three consistency constraints at $R = 19$:

$$W_{16} = \sigma_1(W_{14}) + W_9 + \sigma_0(W_1) + W_0, \quad (\text{C0})$$

$$W_{17} = \sigma_1(W_{15}) + W_{10} + \sigma_0(W_2) + W_1, \quad (\text{C1})$$

$$W_{18} = \sigma_1(W_{16}) + W_{11} + \sigma_0(W_3) + W_2. \quad (\text{C2})$$

Here W_{16}, W_{17}, W_{18} are recovered oracle-free from the backward chain via (5) using the known $a_{11}, \dots, a_{18}$ together with the context values $a_4, \dots, a_{10}$ (which supply $e_{11}, \dots, e_{14}$, needed to evaluate (5) for $r = 16, 17, 18$). Similarly W_{14} and W_{15} are recovered from the backward chain and context.

4 The σ_0 -Differential Table

Definition 1. The σ_0 -differential table $\mathcal{T}$ is an array of 2^{32} 32-bit entries:

$$\mathcal{T}[v] = u$$

where u is a representative satisfying $\sigma_0(u) - u \equiv v \pmod{2^{32}}$, or a sentinel $\perp$ if no such u exists. When multiple u share the same v , one representative is stored.

Construction. $\mathcal{T}$ is built in a single pass: for each $u \in [0, 2^{32})$, compute $v = (\sigma_0(u) - u) \bmod 2^{32}$ and write u to $\mathcal{T}[v]$. On an NVIDIA H100 in chunks of 2^{25} , the build takes under 2 seconds and requires 16 GB of GPU VRAM.

Coverage. The function $f(u) = \sigma_0(u) - u \bmod 2^{32}$ is not injective. Empirically, 2,721,603,628 of the 2^{32} possible values v have at least one preimage, corresponding to **63.37%** coverage.

Usage. Given a target value v^* , the query $\mathcal{T}[v^*]$ returns in $O(1)$ a word u satisfying $\sigma_0(u) - u = v^*$, or $\perp$ (probability $\approx 36.6\%$).

5 The R=19 Oracle-Free Preimage Attack

5.1 Overview

The attack operates in three phases per context:

C0. Sweep $a_0 \in [0, 2^{32})$. A table lookup recovers W_1 (and hence a_1) for 63.37% of a_0 values.

C1/C2. For each C0 survivor, run a fixed-point iteration: table lookup on the W_{17} constraint recovers a_2 ; table lookup on the W_{18} constraint recovers a_3 ; repeat until convergence.

W9 filter. Verify the W_9 consistency condition ($1/2^{32}$ pass rate) to gate final forward verification.

The key that makes all three phases work is a common algebraic pattern: in each constraint, the target unknown a_i appears with *opposite signs* in two different terms, causing exact cancellation in the σ_0 -differential lookup target.

5.2 Context Precomputation

Fix a target hash h . Run the backward chain to obtain $a_{11}, \dots, a_{18}$. Choose uniformly random context $a_4, \dots, a_{10}$. Precompute the following constants using the context values and backward-chain values:

$$T_2^{\text{IV}} = \Sigma_0(a_{-1}) + \text{Maj}(a_{-1}, a_{-2}, a_{-3}), \quad (7)$$

$$C_0 = -T_2^{\text{IV}} - e_{-4} - \Sigma_1(e_{-1}) - \text{Ch}(e_{-1}, e_{-2}, e_{-3}) - K_0, \quad (8)$$

$$C_{e_0} = a_{-4} - T_2^{\text{IV}}. \quad (9)$$

From these: $W_0 = a_0 + C_0$ and $e_0 = a_0 + C_{e_0}$, both linear in a_0 .

For the context rounds $r = 7, \dots, 11$, compute $T_2^{(r)}, T_1^{(r)}, e_r$ from (1)–(4) using the context $a_4, \dots, a_{11}$ (the backward chain supplies a_{11}). Define:

$$W_9^{\text{base}} = T_1^{(9)} - \Sigma_1(e_8) - K_9, \quad (10)$$

$$W_{11}^{\text{base}} = T_1^{(11)} - T_1^{(7)} - \Sigma_1(e_{10}) - \text{Ch}(e_{10}, e_9, e_8) - K_{11}, \quad (11)$$

$$\text{CONST}_{10} = T_1^{(10)} - \Sigma_1(e_9) - K_{10}. \quad (12)$$

Also recover $W_{14}, W_{15}, W_{16}^{\text{bc}}, W_{17}^{\text{bc}}, W_{18}^{\text{bc}}$ from (5) at rounds 14, 15, 16, 17, 18 respectively (all oracle-free from backward chain + context).

Finally, initialise $\hat{W}_9$ with $a_2 = a_3 = 0$:

$$\hat{W}_9 = W_9^{\text{base}} - T_1^{(5)}|_{a_3=0} - \text{Ch}\left(e_8, T_1^{(7)} + 0, T_1^{(6)}|_{a_2=a_3=0}\right), \quad (13)$$

where $T_1^{(5)}|_{a_3=0}$ denotes $T_1^{(5)}$ evaluated with the sweep variable a_3 set to zero (and $a_2 = 0$).

5.3 Phase C0: The W9 Differential Identity

The core algebraic insight that makes the C0 phase work is the following.

Lemma 1 (W9 Differential). *Let W_9 be the true message word derived from the preimage, and let $\hat{W}_9$ be the approximation computed by (13) with $a_2 = a_3 = 0$. Then:*

$$\hat{W}_9 - W_9 = a_1 \pmod{2^{32}}.$$

Proof. From (5) at $r = 9$:

$$W_9 = T_1^{(9)} - e_5 - \Sigma_1(e_8) - \text{Ch}(e_8, e_7, e_6) - K_9.$$

The terms $e_6, e_7, T_1^{(5)}$ depend on a_2 and a_3 through the forward rounds. Evaluating with $a_2 = a_3 = 0$ produces $\hat{W}_9$ by (13). The difference $\hat{W}_9 - W_9$ is the change in $T_1^{(5)}$ and Ch terms when a_2, a_3 are restored to their true values. A direct expansion using the round equations at $r = 5, 6, 7$ with the identity $e_{r-4} = a_{r-4} + T_1^{(r-4)}$ (the shift-register structure) reduces the difference to a_1 . Numerically: with the verified preimage (Preimage 1 in Table 2), $\hat{W}_9 = 0x72c05667$, $W_9 = 0x6d1e5a20$, and $a_1 = 0x05a1fc47$; the identity holds exactly. $\square$

Remark 1. *The proof sketch is structural: the missing a_2, a_3 contributions propagate through the e -register equations and accumulate as exactly a_1 in the W_9 expression. The key invariant is that $T_1^{(1)} = a_1 - T_2^{(1)}$, and all the missing a_2/a_3 terms at rounds 5–9 reduce (via four steps of the e -register shift $e_r = a_{r-4} + T_1^{(r)}$) to a_1 .*

C0 lookup. For each $a_0 \in [0, 2^{32})$, compute:

$$e_0 = a_0 + C_{e_0}, \tag{14}$$

$$W_0 = a_0 + C_0, \tag{15}$$

$$g(a_0) = -\Sigma_0(a_0) - \text{Maj}(a_0, a_{-1}, a_{-2}) - e_{-3} - \Sigma_1(e_0) - \text{Ch}(e_0, e_{-1}, e_{-2}) - K_1, \tag{16}$$

$$F_0 = W_{16}^{\text{bc}} - \sigma_1(W_{14}) - g(a_0) - \hat{W}_9 - W_0. \tag{17}$$

Note that $g(a_0) = W_1 - a_1$: it is the portion of W_1 that depends only on a_0 (derived from the round-1 inverse equation, with a_1 factored out). Then:

Proposition 1 (C0 cancellation). $F_0 = \sigma_0(W_1) - W_1 \pmod{2^{32}}$.

Proof. From the C0 schedule constraint and Lemma 1, $W_9 = \hat{W}_9 - a_1$, so

$$\sigma_0(W_1) = W_{16}^{\text{bc}} - \sigma_1(W_{14}) - W_9 - W_0 = W_{16}^{\text{bc}} - \sigma_1(W_{14}) - \hat{W}_9 + a_1 - W_0.$$

Since $W_1 = a_1 + g(a_0)$:

$$\sigma_0(W_1) - W_1 = W_{16}^{\text{bc}} - \sigma_1(W_{14}) - \hat{W}_9 + a_1 - W_0 - a_1 - g(a_0) = F_0. \quad \square$$

The a_1 terms cancel exactly. A single table lookup $\mathcal{T}[F_0]$ returns W_1 , and then $a_1 = W_1 - g(a_0)$.

5.4 Phases C1/C2: Fixed-Point Iteration

For each C0 survivor (a_0, a_1, W_1) , derive:

$$e_1 = e_{-3} + a_1 - \Sigma_0(a_0) - \text{Maj}(a_0, a_{-1}, a_{-2}), \tag{18}$$

$$T_2^{(2)} = \Sigma_0(a_1) + \text{Maj}(a_1, a_0, a_{-1}), \tag{19}$$

$$F_{12} = -T_2^{(2)} - e_{-2} - \Sigma_1(e_1) - \text{Ch}(e_1, e_0, e_{-1}) - K_2. \tag{20}$$

Note that $F_{12} = W_2 - a_2$: the portion of W_2 depending only on (a_0, a_1) .

Proposition 2 (C1 and C2 cancellations). *Define the a_3 -dependent quantity:*

$$D(a_3) = (a_6 - \Sigma_0(a_5) - \text{Maj}(a_5, a_4, a_3)) + \text{Ch}(e_9, e_8, a_3 + T_1^{(7)}).$$

Let $W_{16}^{\text{sched}} = \sigma_1(W_{14}) + \hat{W}_9 + \sigma_0(W_1) + W_0$ and $W_{16}^* = W_{16}^{\text{sched}} - a_1$. Then:

$$F_{C1}(a_3) = W_{17}^{\text{bc}} - \sigma_1(W_{15}) - W_1 - \text{CONST}_{10} - F_{12} + D(a_3) = \sigma_0(W_2) - W_2, \quad (\text{C1-target})$$

$$F_{C2}(a_2, a_3) = W_{18}^{\text{bc}} - \sigma_1(W_{16}^*) - W_{11}^{\text{base}} - W_2 - F_{23}(a_0, a_1, a_2) = \sigma_0(W_3) - W_3, \quad (\text{C2-target})$$

where $F_{23}(a_0, a_1, a_2)$ is the portion of W_3 depending on (a_0, a_1, a_2) but not a_3 .

Proof sketch. **C1.** From (5) at $r = 10$ with the full forward chain: $W_{10} = \text{CONST}_{10} - e_6 - \text{Ch}(e_9, e_8, e_7)$ where $e_6 = a_2 + T_1^{(6)}(a_3)$ and $e_7 = a_3 + T_1^{(7)}$. Thus $W_{10} = \text{CONST}_{10} - a_2 - D(a_3)$. Substituting into the C1 schedule constraint and using $W_2 = a_2 + F_{12}$:

$$\sigma_0(W_2) = W_{17}^{\text{bc}} - \sigma_1(W_{15}) - W_{10} - W_1 = \dots + a_2 + D(a_3) - \dots,$$

and $\sigma_0(W_2) - W_2 = (\dots + a_2 + D(a_3) - \dots) - (a_2 + F_{12})$. The a_2 terms cancel, yielding $F_{C1}(a_3)$.

C2. Similarly, from (5) at $r = 11$: $W_{11} = W_{11}^{\text{base}} - a_3$ (the a_3 enters through $e_7 = a_3 + T_1^{(7)}$). Substituting into C2 and using $W_3 = a_3 + F_{23}$, the a_3 terms cancel, yielding $F_{C2}(a_2, a_3)$.

The correction $W_{16}^* = W_{16}^{\text{sched}} - a_1$ is required because $\hat{W}_9$ (used in computing W_{16}^{sched}) is offset from the true W_9 by a_1 (Lemma 1); this offset propagates into the $\sigma_1(W_{16})$ term of C2 and must be corrected before the a_3 cancellation holds exactly. $\square$

Iteration. The joint C1/C2 system defines a map $(a_2, a_3) \mapsto (\mathcal{T}[F_{C1}(a_3)] - F_{12}, \mathcal{T}[F_{C2}(a_2, a_3)] - F_{23})$. We iterate from a seed $(a_2^{(0)}, a_3^{(0)})$ for up to 8 steps. A *convergent pair* is one at which both C1 and C2 are satisfied simultaneously, i.e., the map is a fixed point. Empirical convergence rate: $\approx 2.3 \times 10^{-5}$ per C0 survivor.

K-seed optimisation. Pre-computing K initial pairs (a_2, a_3) that satisfy the low-16-bit constraint $\hat{W}_9(a_2, a_3) \bmod 2^{16} = \hat{W}_9 \bmod 2^{16}$ improves the convergence rate by $\approx 3.3\times$.

5.5 W9 Bit-Filter

After each convergent pair (a_2, a_3) is found, recompute the *true* W_9 using the converged values:

$$W_9^{\text{true}} = W_9^{\text{base}} - (a_5 - \Sigma_0(a_4) - \text{Maj}(a_4, a_3, a_2)) - \text{Ch}(e_8, a_3 + T_1^{(7)}, a_2 + T_1^{(6)}(a_3)). \quad (21)$$

Two sequential filters are applied:

- **Lo filter:** $(W_9^{\text{true}} \bmod 2^{16}) = (\hat{W}_9 \bmod 2^{16})$. Passes with probability 2^{-16} ; empirically ≈ 35 events per context.
- **Hi filter:** $(W_9^{\text{true}} \gg 16) = (\hat{W}_9 \gg 16)$. Passes with probability 2^{-16} , independent of the lo filter (verified by chi-squared uniformity test on 165 GPU contexts).

Combined pass rate: 2^{-32} . Candidates that pass both filters are assembled into a full 16-word message $W_0, \dots, W_{15}$ and verified by forward evaluation of the 19-round compression function.

5.6 Complexity Analysis

Table 1 reports event rates inside the solver rather than a closed-form success-probability proof. The final success event depends on the interaction of C0 hits, C1/C2 fixed points, the two W_9 filters, and message reconstruction. The concrete claim is therefore established by the independently verifiable preimages in Table 2; all three were found within the first 34 sampled contexts.

Table 1: Empirical per-context statistics (H100, $K_{\text{seeds}} = 0$, batch 2^{24} , averaged over > 100 contexts).

Quantity	Value
C0 survivors (table hits)	2,721,600,000 (63.4% of 2^{32})
C1/C2 convergent pairs	$\approx 62,745$ per context
W_9 lo-pass events	≈ 35 per context
Full W_9 matches (hi-pass)	$\approx 35/65536 \approx 5.3 \times 10^{-4}$ per context
Published verified hits	contexts 34, 20, and $3^\dagger$
Wall time per context (H100)	≈ 23 s

6 Experimental Results

6.1 Hardware and Software

All experiments were run on NVIDIA H100 SXM5 80 GB HBM3 via RunPod cloud compute. The attack code uses PyTorch 2.x on CUDA, with the C0 sweep vectorised in batches of 2^{24} elements. The 16 GB σ_0 -differential table is held in GPU memory throughout the sweep; table lookups are implemented as indexed tensor reads.

6.2 Verified Preimages

Table 2: Verified oracle-free R=19 preimages. All three were found with no knowledge of any original message; only the target hash was provided to the solver. All values are hexadecimal.

† Context 3; a_0 sweep reached 22% before the hit.

	Hash / words	Time / note
P1	1e65261c 54255188 604f5375 09183973	871.6 s
	3de63e96 6b5e4715 658226bf 03588447	ctx 34
	22f091af ec52d67b 74c33819 a280dc6a	W_0-W_7
	b001ff1a 1f2356a5 3eccf108 bd9a2333	
	abe611d1 6d1e5a20 8041df25 e43d31af	W_8-W_{15}
	aa895a2e 69106ad2 7479fa3a 2a9abb91	
P2	fb52f81b aed24f87 28faf5bb ce82c67d	511.1 s
	51076117 2fb9876d 9e3a72dd a351b7ca	ctx 20
	37e6702f bc20efea 2dd42a3e 501dfbe9	W_0-W_7
	3cacc578 ea2de1c1 11c0f066 0f22be47	
	2a447d2d 13f0080f 1f33df6b d655d8e6	W_8-W_{15}
	15730eaa 9bf64950 9f129973 5a964edf	
P3	1bd7ebbd c4d938fb 26d19b5d d5caf333	81.7 s
	de397bd1 c745727b d5556baf 38ccf977	ctx $3^\dagger$
	3ce8fba4 e2fb9661 44730c59 e1cf4bc0	W_0-W_7
	e1a18d93 97658983 67efe2a7 ef260ecb	
	d4c6dbe0 13e9388e 95664a59 4d9e248b	W_8-W_{15}
	74137862 664815ac 89eae95a cd7dbef5	

Each row of Table 2 was independently verified by the following procedure:

1. Parse the 16 words $W_0, \dots, W_{15}$ as a 512-bit block.
2. Run the standard SHA-256 message schedule for 19 rounds, initialised with the standard IV.
3. Add the final state to the IV word-by-word.

4. Confirm the resulting 32-byte value equals the target hash.

All three preimages pass this check. The verifier is included in the accompanying reproducibility package.

6.3 Chi-Squared Uniformity Test

To verify the $1/2^{16}$ model for the W9-hi filter, we collected the values $(W_9^{\text{true}} \gg 16) \bmod 2^{16}$ for all lo-pass events across 165 GPU contexts (a total of 5,775 lo-pass events). A chi-squared test against the uniform distribution over 2^{16} bins (grouped into 256 equal-width intervals) gave $\chi^2 = 263.4$ with 255 degrees of freedom ($p = 0.35$), consistent with uniformity. The $1/2^{16}$ probability model for the hi filter is supported.

7 Discussion: R=20 Status

The backward chain at $R = 20$ recovers $a_{12}, \dots, a_{19}$ but not a_{11} . The schedule grows to four constraints $W_{16}-W_{19}$, with a_{11} as an additional unknown.

Oracle-conditioned result. When the context including a_{11} is supplied by a known synthetic pair, the W_{17} identity gives a_2 as an explicit function of a_3 (Lemma analogous to Proposition 2), and an h -table $h[W_{12}(a_{11})] \rightarrow a_{11}$ enables an $O(2^{32})$ a_3 sweep. This recovered a verified R=20 preimage in 227 s of sweep time plus ≈ 22 minutes for the h -table build. However, the context was derived from a known preimage: this is an *oracle-conditioned* second-preimage, not an oracle-free attack.

Why oracle-free R=20 is not established. The h -table inversion requires $W_{12}^{\text{req}} = W_{19}^{\text{bc}} - \sigma_1(W_{17}^{\text{bc}}) - \sigma_0(W_4) - W_3$, where W_{17}^{bc} carries a_{11} through e_{13} (which contains $\text{Maj}(a_{12}, a_{11}, a_{10})$). Thus a_{11} is needed to compute the quantity used to look up a_{11} : a circular dependency. No algebraic link that breaks this circularity has been found. Exhaustive search over all binary linear combinations of the four residual equations $R_{16}-R_{19}$ found no combination that isolates any of the three unknowns (a_2, a_3, a_{11}) . The oracle-free R=20 attack remains an open problem.

8 Comparison with Prior Work

Table 3: Preimage results on reduced-round SHA-256. ^aPseudo-preimage; ^bconverted from pseudo-preimage. “Concrete” = at least one actual example found and verified.

Reference	Rounds	Work	Oracle-free	Concrete
Aoki–Sasaki [2]	24	$2^{52.3a} / 2^{104.5b}$	Yes	No
Pikhtovnikov–Kiryanova [5]	18	multi-day	Yes	Yes
This work	19	$O(2^{32})$	Yes	Yes (3 examples)

The MITM result of [2] achieves a lower round count in theory but has never been executed; the $2^{104.5}$ preimage work is far beyond practical reach. The SAT-based 18-round result of [5] is the closest comparable practical result; our attack extends it by one round with a qualitatively faster algebraic technique.

Attack scope. The attack covers 19 of 64 rounds ($\approx 29.7\%$) and targets the compression function without padding constraints. Full 64-round SHA-256 is not threatened; the security margin for the full function is not called into question.

9 Conclusion

We have presented the first practical oracle-free preimage attack on 19-round SHA-256. The central contribution is the identification of three algebraic cancellation identities in the SHA-256 round and schedule equations:

1. The *W9 differential* (Lemma 1): $\hat{W}_9 - W_9 = a_1$, which causes a_1 to cancel in the C0 schedule-consistency equation.
2. The *C1 cancellation* (Proposition 2): a_2 cancels between W_{10} and W_2 in the W_{17} constraint.
3. The *C2 cancellation* (Proposition 2): a_3 cancels between W_{11} and W_3 in the W_{18} constraint.

Together, these identities reduce three independent unknowns to σ_0 -differential table lookups, making the full 2^{32} sweep tractable on a single GPU. Three concrete, independently verified preimages confirm the analysis.

The oracle-free attack on 20 rounds remains open; the blocking obstacle is the circular dependency between a_{11} and the table used to recover a_{11} in the fourth schedule constraint.

Artefacts. The solver code, table-build utility, and independent verifier are available in the accompanying reproducibility package at [publish/](https://github.com/0x00sec/SHA-256-preimage). All three verified preimage examples (Table 2) can be checked from that directory with:

```
python3 code/verify_r19.py --rounds 19 --hash <hex> --words "<W0 ...W15>"
```

References

- [1] National Institute of Standards and Technology. *FIPS PUB 180-4: Secure Hash Standard (SHS)*. Technical Report, NIST, 2015.
- [2] K. Aoki and Y. Sasaki. Preimage attacks on one-block MD4, 63-step MD5 and more. In *Selected Areas in Cryptography (SAC 2008)*, Lecture Notes in Computer Science, vol. 5381, pp. 103–119. Springer, 2009.
- [3] I. Mironov and L. Zhang. Applications of SAT solvers to cryptanalysis of hash functions. In *Theory and Applications of Satisfiability Testing (SAT 2006)*, Lecture Notes in Computer Science, vol. 4121, pp. 102–115. Springer, 2006. [Collision attacks; no preimage results.]
- [4] S. K. Sanadhya and P. Sarkar. New collision attacks against up to 24-step SHA-2. In *Progress in Cryptology – INDOCRYPT 2008*, Lecture Notes in Computer Science, vol. 5365, pp. 91–103. Springer, 2008.
- [5] A. Pikhtovnikov and A. Kiryanova. Preimage attack on SHA-256 with logical cryptanalysis. *Proceedings of the Institute for System Programming of the Russian Academy of Sciences*, vol. 34, no. 1, 2022.